

LABORATOR 3

Sisteme de Operare Procese

1. Scopul lucrării

Pentru a înțelege și stăpâni Unix-ul este foarte importanta noțiunea de proces. Procesul sta la baza oricărei activități din sistemul de operare. La lansarea unei comenzi utilizator se creează și un nou proces. Controlul proceselor este un element important în programarea pe sisteme multitasking. Ea include operații de creare de procese, terminare și sincronizare. Sistemul Unix este și un sistem multi-utilizator acest lucru fiind un aspect important în dezvoltarea aplicațiilor multiutilizator. Controlul proceselor este realizat prin câteva apeluri sistem, care nu sunt altceva decât funcții, înglobate în nucleu, accesibile utilizatorului. Utilizarea corectă a apelurilor este esențială în controlul corect al proceselor Unix.

2. Considerații teoretice

Un proces este instanța execuției unui program. Procesele nu se confunda cu programul, care este fișierul executat de proces. Pe un sistem multitasking mai multe procese pot executa același program concurrent și fiecare proces se poate autotransforma pentru a executa un program anume.

Fiecare proces în Unix are asociat un identificator unic numit *identicator de proces*, prescurtat PID. Pentru identificator se utilizează în continuare prescurtarea ID. PID este un număr pozitiv atribuit de sistemul de operare fiecărui proces nou creat. Un proces poate să determine PID-ul sau folosind apelul sistem **getpid**. Cum PID-ul unui proces este unic el nu poate fi schimbat, dar se poate refolosi când procesul nu mai există.

Apel	Acțiune
int getpid()	Returnează ID procesului apelant;
int getpgrp()	Returnează ID grupului de procese;
int getppid()	Returnează ID procesului părinte;

Tab.1. Apeluri care întorc identificatori de proces.

Orice proces nou în Unix este creat de un proces anterior existent, dând naștere unei relații părinte-fiu. Excepție face procesul 0, care este creat și utilizat chiar de nucleu. Un proces poate să determine PID-ul părintelui prin apelul **getppid()**. PID-ul procesului părinte nu se poate modifica.

Sistemul de operare Unix tine evidenta proceselor intr-o structura de date interna numita *tabela de procese*. Ea are o intrare pentru fiecare proces din sistem. Lista proceselor din tabela de procese poate fi obtinuta prin comanda **ps**.

Uneori se doreste crearea unui subsistem ca un grup de procese înrudite în locul unui proces singular. De exemplu, un sistem complex de gestiune al unei baze de date poate fi împărțit în câteva procese pentru a "câștiga" operații de I/E cu discul.

2.1. Grup de procese

Pe lângă ID asociat unui proces, care permite identificarea individuala a fiecăruia, fiecare proces are și un ID de grup de procese, prescurtat (PGID), care permite identificarea unui grup de procese. PGID este moștenit de procesul fiu de la procesul părinte. Contrar PID-ului, un proces poate sa-si modifice PGID, dar numai prin crearea unui nou grup. Acest lucru se realizează prin apelul sistem **setpgrp**.

```
int setpgrp();
```

setpgrp actualizează PGID-ul procesului apelant la valoarea PID-ului sau și întoarce noul PGID. Procesul apelant părăsește astfel vechiul grup devenind leaderul propriului grup urmând a-si crea procesele fiu, care sa formeze grupul. Deoarece procesul apelant este primul membru al grupului și numai descendenții săi pot sa aparțină grupului (prin moștenirea PGID), el este referit ca reprezentantul (leaderul) grupului.

Deoarece doar descendenții leaderului pot fi membri ai grupului exista o corelație între grupul de procese și arborele proceselor. Fiecare leader de grup este rădăcina unui subarbore, care după eliminarea rădăcinii conține doar procese ce aparțin grupului. Daca nici un proces din grup nu s-a terminat lăsând fii care au fost adoptați de procesul **init**, acest subarbore conține toate procesele din grup.

Un proces poate determina PGID sau folosind apelul sistem:

```
int getpgrp();
```

Apelul întoarce PGID procesului apelant. Deoarece PID-ul leaderului este același cu PGID-ul, **getpgrp** identifica leaderul.

Un proces poate fi asociat unui terminal, care este numit *terminalul de control* asociat procesului. Acesta este moștenit de la procesul părinte la crearea unui nou proces. Un proces este deconectat (eliberat) de terminalul sau de control la apelul **setpgrp**, devenind astfel un leader de grup de procese (nu se închide terminalul). Ca atare, numai leaderul poate stabili un terminal de control, devenind procesul de control pentru terminalul în cauza.

Rațiunea existentei unui grup este legata de comunicarea prin semnale. în mod uzual, procesele din același grup sunt conectate logic în acest fel. De exemplu, managerul unei baze de date multiproces este constituit dintr-un proces master și câteva procese subsidiare. Pentru a face din

suita proceselor bazei de date un grup de procese, procesul master apelează **setpgid** înainte de a crea procesele subsidiare.

Un proces care nu este asociat cu un terminal de control este numit *daemon*. Spoolerul de imprimantă este un exemplu de astfel de proces. Un proces daemon este identificat în rezultatul afișării comenzii **ps** prin simbolul **? plasat în coloana TTY**.

2.2. Programe și procese

Un program este o colecție de instrucțiuni și date păstrate într-un fișier ordinar pe disc. În fișierul sau fișierul este marcat executabil și conținutul său este aranjat conform regulilor stabilite de nucleu.

Un program în Unix este format din mai multe segmente. În segmentul de cod se găsesc instrucțiuni sub forma binară. În segmentul de date se găsesc date predefinite (de exemplu constante) și date inițializate. Al treilea segment este segmentul de stivă, care conține date alocate dinamic la execuția procesului. Aceste trei segmente sunt și părți funcționale ale unui proces Unix. Pentru a executa un program nucleul este informat pentru a crea un nou proces, care nu este altceva decât un mediu în care se execută un program. Un proces constă din trei segmente: segmentul de instrucțiuni, segmentul de date utilizator și segmentul de date sistem. Programul este folosit pentru a inițializa primele două segmente, după care nu mai există nici o legătură între procesul și programul pe care-l execută. Datele sistem ale unui proces includ informații ca directorul curent, descriptori de fișiere deschise, cai implicite, tipul terminalului, timp CPU, etc. Un proces nu poate accesa sau modifica direct propriile date sistem, deoarece acestea sunt în afara spațiului de adresare. Există însă multiple apeluri sistem pentru a accesa sau modifica aceste informații. Structura arborescentă a sistemului de fișiere implică o structură identică pentru procese. Toate procesele active la un moment dat în Unix sunt de fapt descendenți direcți sau indirecti ai unui singur proces, lansat la pornirea sistemului prin comanda **/etc/init**.

După pornirea sistemului, printr-un dialog la consola se poate opta pentru trecerea în mod multiutilizator. În acest caz, sistemul citește din fișierul **/etc/ttys** numerele terminalelor utilizator. Pentru fiecare dintre aceste terminale va fi lansat un nou proces. Procesul de la un terminal precum și urmașii lui vor avea intrarea standard, ieșirea standard și ieșirea de erori fixate la terminalul respectiv. Se setează apoi viteza de lucru, se citesc parametrii de comunicație ai terminalelor, paritatea, etc. Utilizatorul introduce apoi numele și eventual parola proprie, care în cazul în care sunt corecte, se lansează un nou proces care nu este altceva decât interpretorul de comenzi shell. Acesta are menirea de a interpreta comenzile utilizator. Dacă lansarea unei comenzi externe în DOS se făcea prin încărcarea în memorie și lansarea în execuție, nu același lucru se poate spune despre o comandă Unix. Într-un sistem multitasking și multiuser la lansarea unui program se creează de fiecare dată un nou proces. Acest lucru se realizează prin apelul sistem **fork**. La fiecare execuție a acestui apel, se obțin două procese concurente, identice la început, dar cu nume diferite. Apelul sistem **fork** realizează o copie a procesului inițial, ca atare imaginea proceselor în memorie este identică. Procesul care a inițiat apelul **fork** este identificat ca proces părinte sau tată, iar procesul rezultat în urma apelului este identificat ca proces fiu. De exemplu, se considera comanda: **\$echo Exemplu**

Shell-ul desparte comanda de argumente. Se executa apelul **fork** și rezulta procesul fiu. Procesul tata prin apelul sistem **wait** cedează procesorul procesului fiu. Procesul fiu cere nucleului, prin apelul sistem **exec**, execuția unui nou program, respectiv **echo** și comunica în același timp și argumentele pentru noul program. Nucleul eliberează zona alocata pentru shell și încărca un nou program pentru procesul fiu. Procesul continua cu execuția noului program. Execuția lui **exit** are ca efect terminarea procesului curent (fiu), ștergerea legaturilor și transmiterea unui cod de terminare procesului tata, care părăsește astfel starea de așteptare și înlătura procesul fiu din sistem. Este posibila execuția unei comenzi într-un plan secundar (background), daca după specificarea comenzii este adăugat caracterul **&**. La o astfel de comanda interpretorul răspunde cu un număr după care afișează prompterul. Acest număr reprezintă ID de proces al comenzii introduse. Procesul shell care lansează comanda nu mai așteaptă încheierea procesului fiu. Acesta din urma se va executa independent de procesul care l-a creat, iar după apelul **exit** el rămâne în stare inactiva în sistem pana când shell-ul va executa un apel **wait** în urma căruia procesul este înlăturat din sistem.

2.3. Apelurile sistem FORK și EXEC

Apelul sistem **fork** are sintaxa:

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t fork( void);
```

Returneaza 0 în procesul fiu, PID fiului în procesul tata și -1 în caz de eroare.

Figura de mai jos ilustrează cum se obține prin **fork o copie identica a procesului tata.**

	Procesul tata	Procesul fiu
...		...
pid=fork();	fork()	pid=fork();
if (pid==0) {	----->	if (pid==0) {
/* fiu */	start -->	/* fiu */
} else { /* tata */ }		} else { /* tata */ }

Fig.1. Generarea unui nou proces prin fork

În noul proces (fiu) toate vechile variabile își păstrează valorile, toți descriptorii de fișier sunt aceiași, se moștenește același UID real și GUID real, același ID de grup de procese, aceleași variabile de context. Bineînțeles ca noua copie a procesului tata se găsește fizic la o alta adresa de memorie. Din momentul revenirii din apelul **fork**, procesele tata și fiu se executa independent, concurând unul cu celalalt pentru obținerea resurselor. Procesul fiu își începe

execuția din locul unde rămăsese procesul tata. Nu se poate preciza care dintre procese va porni primul. Este posibilă însă separarea execuției în cele două procese prin testarea valorii întoarse de apelul **fork**. Secvența de mai jos partiționează codul programului în cele două procese:

```
...

if ((pid=fork())==0)/* acțiuni specifice fiului */

else if ( pid!=0) /* acțiuni specifice tatălui */

else /* eroare la apelul fork */
```

Cazul de eroare poate apărea dacă s-a atins limita maximă de procese pe care le poate lansa un utilizator sau dacă s-a atins limita maximă de procese ce se pot executa deodată în sistem.

Rățiunea a două procese identice are sens dacă se poate modifica segmentul de date și cel de cod al procesului rezultat așa încât să se poată încărca un nou program. Pentru acest lucru există apelul **exec** (împreună cu familia de astfel de apeluri **execl**, **execp**, **execv** și **execvp**). **Partea de sistem a procesului nu se modifică în nici un fel prin apelul exec**. Deci nici numărul procesului nu se schimbă. Practic procesul fiu execută cu totul altceva decât părintele sau. După un apel **exec** reușit nu se mai revine în vechiul cod. A nu se uita că fișierele deschise ale tatălui se regăsesc deschise și la fiu după apelul **exec** și că indicatorul de citire/scriere al fișierelor deschise rămâne nemodificat, ceea ce poate cauza neplăceri în cazul în care tatăl și fiul vor să scrie în același loc. Un apel **exec** nereușit returnează valoarea -1, dar cum alta valoare nu se returnează ea nu trebuie testată. Insuccesul poate fi determinat de cale eronată sau fișier neexecutabil. Diferitele variante de **exec** dau utilizatorului mai multă flexibilitate la transmiterea parametrilor. Sintaxele lor sunt:

```
#include <unistd.h>

(1) execl ( const char *path, const char *arg0, ..., NULL);

(2) execv ( const char *path, char *argv[]);

(3) execlp( const char *filename, const char *arg0, ..., NULL);

(4) execvp( const char *filename, char *argv[]);
```

Returnează -1 în caz de eroare, nimic în caz de succes.

Între apelurile 1) și 3) respectiv între 2) și 4) există două deosebiri de implementare. Prima este că al ultimelor două apeluri căută programul pe calele implicite setate de variabila **PATH** și se pot lansa și proceduri shell. Dacă *filename* nu conține caractere slash se expandează numele fișierului cu calea găsite în variabila **PATH** și se încearcă găsirea unui fișier executabil care să se potrivească cu calea obținută. Dacă nu se găsește nici un astfel de fișier se presupune că este o comandă shell și se execută **/bin/sh**. Dacă în cale se precizează caractere slash se presupune calea completă și nu se face nici o căutare.

A doua deosebire se refera la transmiterea argumentelor ca lista sau vector. în cazul listelor fiecare argument este precizat separat și sfârșitul listei este marcat printr-un pointer NULL. în cazul vectorului se specifica doar adresa unui vector cu adresele argumentelor. Mai aproape de modul obișnuit de lucru este apelul 2). Se folosește des în momentul compilării când nu se cunoaște numărul de argumente.

Se observa ca fără **fork**, **exec** este limitat ca acțiune, iar fără **exec**, **fork** nu are aplicabilitate practica. Deși efectul lor conjugat este cel dorit, rațiunea existenței a doua apeluri distincte va rezulta din parcurgerea lucrărilor următoare.

2.4. Sincronizare tata-fiu. Apelul sistem WAIT și WAITPID

Procesul tata poate aștepta terminarea (normala sau cu eroare) procesului fiu folosind apelul sistem **wait** sau **waitpid**.

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait( int *pstatus);
pid_t waitpid(pid_t pid, int *pstatus, int opt);
```

Returneaza PID în caz de succes sau 0 (*waitpid*), -1 în caz de eroare.
Argumentul *pstatus* este adresa cuvântului de stare.

Un proces ce apelează **wait** sau **waitpid** poate:

- sa se blocheze (daca toți fiii săi sunt în execuție),
- sa primească starea de terminare a fiului (daca unul dintre fii s-a terminat),
- sa primească eroare (daca nu are procese fiu).

Diferențele între cele doua funcții constau în:

a) **wait** blochează procesul apelant pana la terminarea unui fiu, în timp ce **waitpid** are o opțiune, precizata prin argumentul *opt*, care evita acest lucru.

b) **waitpid** nu așteaptă terminarea primului fiu, ci poate specifica prin argumentul *opt* procesul fiu așteptat.

c) **waitpid** permite controlul programelor prin argumentul *opt*.

Daca, nu exista procese fiu, apelul **wait** întoarce valoarea -1 și poziționează variabila **errno** la **ECHILD**.

În ce mod s-a terminat procesul fiu, normal sau cu eroare, se poate afla cu ajutorul parametrului *pstatus*. Exista cazurile:

Procesul fiu	Procesul tata: <i>pstatus</i>	
oprit		motiv 0177
terminat cu exit(nr)	nr	0000
terminat cu semnal	0000	nr_sem
terminat cu semnal și vidaj de memorie	0200	nr_sem

Tab.2. Valorile returnate prin parametrul *pstatus*.

Exista trei moduri de a termina un proces: apelul **exit**, receptionarea unui semnal fatal, sau caderea sistemului. Codul de stare returnat prin variabila *pstatus* indica care din cele

doua moduri a cauzat terminarea (în al treilea mod procesul parinte și nucleul dispar, asa incat *pstatus* nu conteaza).

Funcția **waitpid** exista în SRV4 și 4.3+BSD. Argumentul **opt** poate avea valorile:

Constanta	Descriere
WNOHANG	Apelul nu se blochează daca fiul specificat prin pid nu este disponibil. în acest caz valoarea de retur este 0.
WUNTRACED	Daca implementarea permite controlul lucrărilor, starea fiecărui proces fiu oprit și neraportată este întoarsa. Macroul WIFSTOPPED determina daca valoarea întoarsa corespunde unui proces fiu oprit.

Tab.3. Valorile argumentului *opt*.

Argumentul **opt** poate fi și 0 sau rezultatul unui SAU între constantele simbolice WNOHANG și WUNTRACED.

În funcție *pid*, interpretarea funcției **waitpid** este:

pid==-1 Se așteaptă orice proces fiu (echivalent **wait**).

pid > 0 Se așteaptă procesul **pid**.

pid==0 Se așteaptă orice proces cu ID de grup de proces egal cu cel al apelantului.

pid< -1 Se așteaptă orice proces cu ID de grup de proces egal cu valoarea absoluta a **pid**.

Apelul **waitpid** returnează (-1) daca nu exista proces sau grup de procese cu *pid*-ul specificat sau *pid*-ul respectiv nu este al unui fiu de al sau.

Pentru a analiza starea în care s-a terminat un proces fiu exista trei macroui excluse mutual, toate prefixate de WIF și definite în fișierul **sys/wait.h**. Pe lângă acestea, exista alte macroui pentru determinarea codului de exit, număr semnal, etc. Acestea sunt ilustrate mai jos:

Macro	Descriere
WIFEXITED(status)	Adevărat daca informația de stare, <i>status</i> , provine de la un proces terminat normal. în acest caz se poate folosi: WEXITSTATUS(status) pentru a extrage octetul mai puțin semnificativ.
WIFSIGNALED(status)	Adevărat daca informația de stare, <i>status</i> , provine de la un proces terminat anormal. în acest caz se poate folosi: WTERMSIG(status) pentru a extrage numărul semnalului.
WIFSTOPPED(status)	În SVR4 și 4.3+BSD macroul WCOREDUMP(status) este adevărat daca s-a generat fișier core. Adevărat daca informația de stare, <i>status</i> , provine de la un proces temporar oprit. în acest caz se poate folosi: WSTOPSIG(status) pentru a extrage numărul semnalului care a oprit procesul.

Tab.4. Macrouri pentru determinarea starii.

2.5. Apelul sistem EXIT

Procesul fiu semnaleză terminarea sa tatălui aflat în aşteptare. Pentru aceasta exista apelul **exit** care transmite prin parametrul sau un număr care semnaleză tatălui o terminare normala sau cu eroare. Prin convenţie, un cod de stare 0 semnifica terminarea normala a procesului, iar un cod diferit de zero indica apariţia unei erori. Sintaxa apelului este:

void exit(int status);

Acest apel termina procesul care-l executa cu un cod de stare egal cu octetul mai semnificativ al cuvântului de stare, *status*, şi închide toate fişierele deschise de acesta. După aceea, procesului tata ii este transmis semnalul SIGCLD.

Pentru procesele aflate într-o relaţie părinte-fiul la un apel **exit** sunt esenţiale trei cazuri:

- procesul părinte se termina înaintea procesului fiu;
- procesul fiu se termina înaintea procesului părinte;
- procesul fiu, moştenit de procesul **init**, **se termina**.

Procesul init devine părintele oricărui proces pentru care procesul părinte s-a terminat. Când un proces se termina nucleul parcurge toate procesele active pentru a vedea dacă printre ele exista un proces care are ca părinte procesul terminat. Dacă exista un astfel de proces, pid-ul procesului părinte devine 1 (pid-ul lui **init**). Nucleul garantează astfel ca fiecare proces are un părinte.

Dacă procesul fiu se termina înaintea procesului părinte, nucleul trebuie sa păstreze anumite informaţii (pid, starea de terminare, timp de utilizare CPU) asupra modului în care fiul s-a terminat. Aceste informaţii sunt accesibile părintelui prin apelul **wait** sau **waitpid**. În terminologie Unix un proces care s-a terminat şi pentru care procesul părinte nu a executat **wait** se numeşte *zombie*. În aceasta stare, procesul nu are resurse alocate, ci doar intrarea sa în tabela proceselor. Nucleul poate descarca toata memoria folosita de proces şi inchide fişierele deschise. Un proces *zombie* se poate observa prin comanda Unix **ps** care afiseaza la starea procesului litera 'Z'.

*Dacă un proces care are ca părinte procesul **init** se termina, acesta nu devine *zombie* iarăşi deoarece, procesul **init** apelează una dintre funcţiile **wait** pentru a analiza starea în care procesul a fost terminat. Prin aceasta comportare procesul **init** evita încărcarea sistemului cu procese *zombie*.*

Apelul **_exit** nu goleşte tamponanele de I/E.

2.6. Gestiunea şi planificarea proceselor

Singura situatie in care procesul apelant revine din apelul functiei *exec()* este acela in care operatia nu a putut fi efectuata, caz in care functia returneaza un cod de eroare (-1).

În consecință, lansarea într-un proces separat a unui program de pe disc se face apelând *fork()* pentru crearea noului proces, după care în porțiunea de cod executată de fiu se va apela una din funcțiile *exec()*.

2.4 Funcțiile *system()* și *vfork()*

`int system(const char *cmd)`

Lansează în execuție un program de pe disc, folosind în acest scop un apel *fork()*, urmat de *exec()*, împreună cu *waitpid()* în părinte.

`pid_t vfork()`

Creează un nou proces, la fel ca *fork()*, dar nu copiază în întregime spațiul de adrese al părintelui în fiu. Este folosit în conjuncție cu *exec()*, și are avantajul că nu se mai consumă timpul necesar operațiilor de copiere care oricum ar fi inutile dacă imediat după aceea se apelează *exec()* (oricum, procesul fiu va fi suprascris cu programul luat de pe disc).

2.5 Alte funcții pentru lucrul cu procese

`pid_t getpid()` - returnează PID-ul procesului curent

`pid_t getppid()` - returnează PID-ul părintelui procesului curent

`uid_t getuid()` - returnează identificatorul utilizatorului care a lansat procesul curent

`gid_t getgid()` - returnează identificatorul grupului utilizatorului care a lansat procesul curent

2.6 Gestionarea proceselor din linia de comandă

Sistemul de operare UNIX are câteva comenzi foarte utile care se referă la procese:

- **ps** - afișează informații despre procesele care rulează în mod curent pe sistem
- **kill -semnal proces** - trimite un semnal unui proces. De exemplu, comanda `kill -9 123` va termina procesul cu numărul 123
- **killall -semnal nume** - trimite semnal către toate procesele cu numele *nume*

Există și alte comenzi utile; pentru folosirea lor, este recomandat să se consulte paginile de manual UNIX.



Consultati paginile de manual corespunzătoare acestor funcții.